

IoT システム設計 第 8 回レポート (出題日: 6/4, 提出日 6/10)

情報理工学科 3 年 1W243015 伊藤叶人

情報理工学科 3 年 1W243021 上治謙二郎

0. 概要 (担当: 伊藤)

グループワークで作成するシステムの仕様書(第 8 回レポート)を共同で記述した。各々が記述した部分は(担当:)という表記で示している。

1. 企画 (担当: 伊藤)

FPGA ボードを入力、ターミナルを出力とした 2 人プレイ用のチェスを作成する。

2. 基本仕様

2.1 要求の整理 (担当: 伊藤)

FPGA 上に存在する入力手段を用いてチェスにおいて必要となる操作を実現する。また、その操作を反映し、実際にチェスというゲームが成立するためのソフトを制作する。その他に、ゲームとしての完成度を高めるため時間制限などのシステムを構築する必要がある。

2.2 入力と出力 (担当: 伊藤)

入力:

- ・操作する駒の選択/駒の移動先の選択 を切り替えるスイッチ
- ・盤上の場所を選ぶスティック
- ・場所の選択を決定するボタン
- ・操作を終了するボタン
- ・タイマー設定などその他の詳細設定を変化させるスイッチ
- ・ゲームを終了/リセットするボタン

出力:

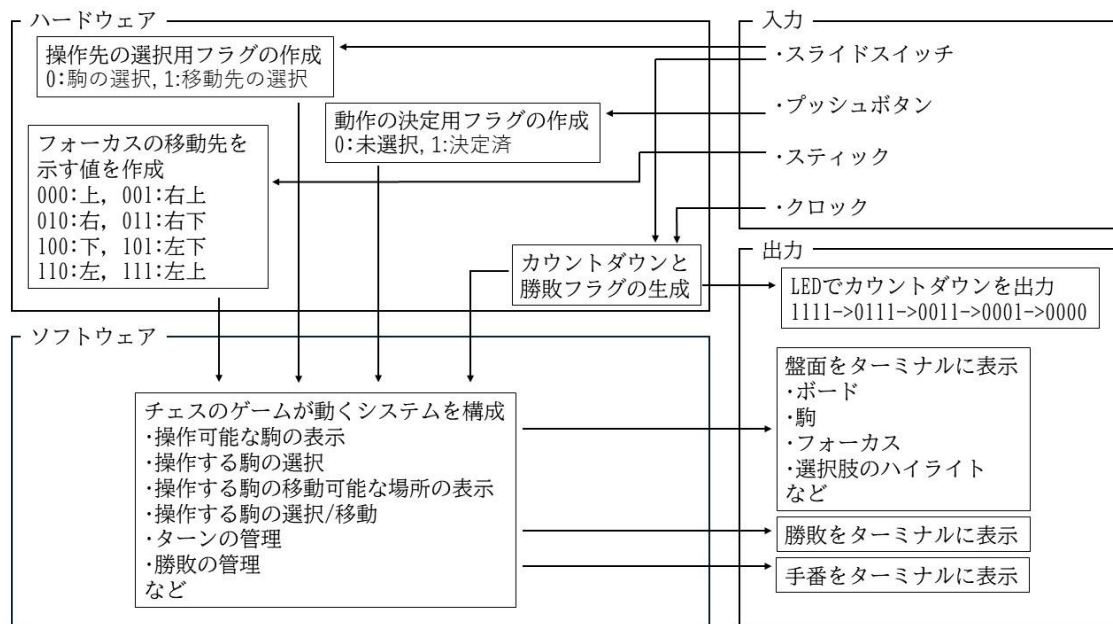
- ・実際のゲームの盤面
- ・タイマー
- ・勝敗の表示

2.3 ハード/ソフトの切り分け (担当: 伊藤)

ハード: 2.3 における入力全てと出力のタイマー

ソフト: 2.3 におけるタイマーを除く出力全て

2.4 システム全体のアーキテクチャ（担当：伊藤）



3. 詳細仕様

3.1 ハードウェア（担当：伊藤）

3.1.1 モジュールの切り分け

・main モジュール

入力を受け取り、他のモジュールを介して出力を計算し、出力をソフトウェアに返す。

・LED モジュール

クロックを用いてタイマーを作成し、カウントの残りを LED を用いて表示する。カウントが 0 になったタイミングでフラグを返す。また、スライドスイッチの入力を受け取り、タイマーのカウントダウンスピードなどを変更する。

・スライドスイッチモジュール

スライドスイッチの入力を受け取り、各種設定や操作する対象の変更をするためのフラグを作成し、出力する。

・プッシュボタンモジュール

ボタンの入力を受け取り、操作の決定やゲームのリセットなどを行えるようにフラグを出力する。

- ・スティックモジュール

スティックの入力を受け取り、その値を変換することで、8方向の操作を表す値を出力する。

3.1.2 入出力の信号

- ・入力信号：クロック、スライドスイッチ、ボタン、スティック
- ・出力信号：時間制限フラグ、操作フラグ、カウントダウン(LED)

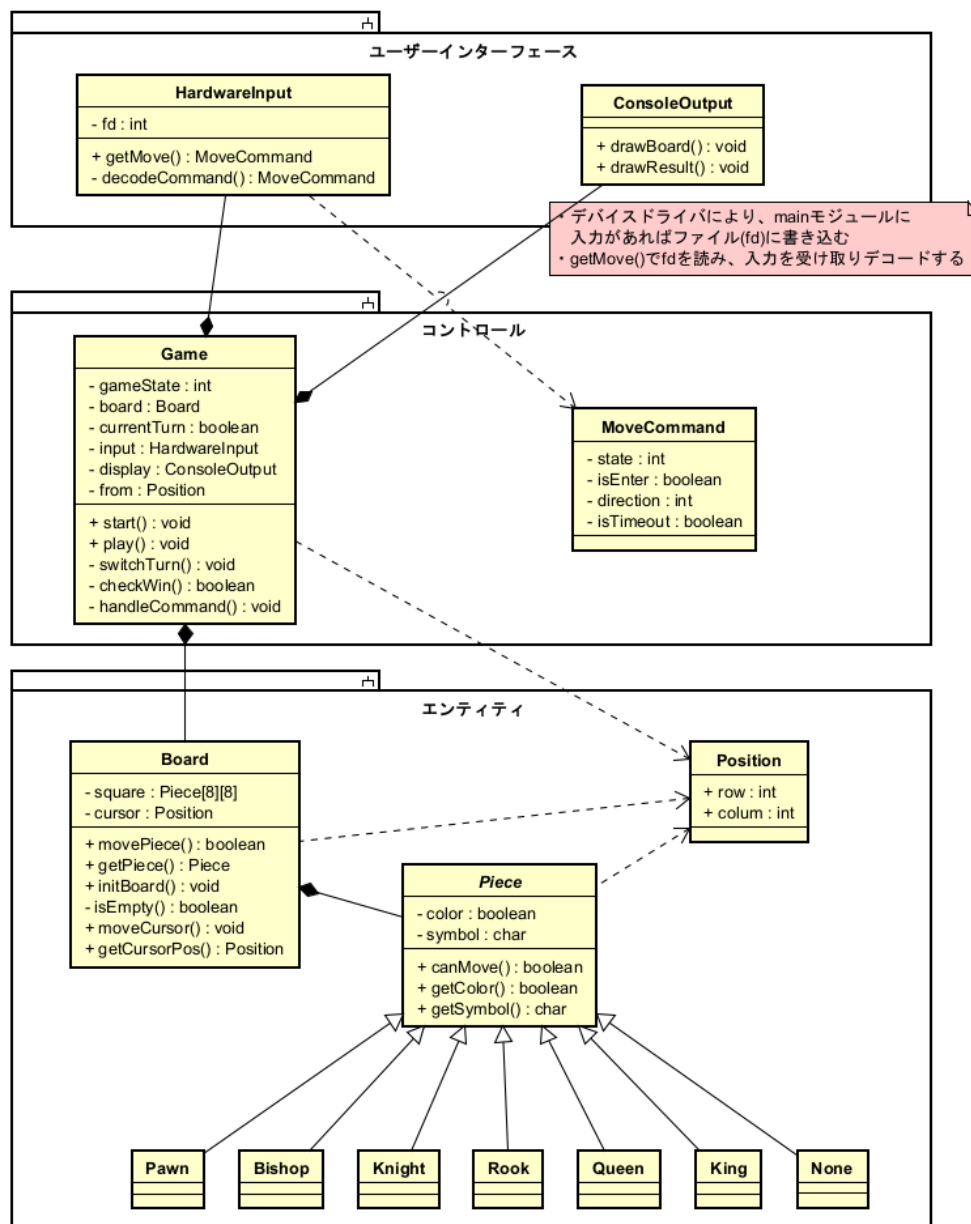
3.2 ソフトウェア（担当：上治）

3.2.1. 機能要件

- ・入力の受付（選択状態、決定ボタン、移動方向、時間切れフラグ）
- ・盤面の表示
- ・ターンの管理
- ・状態の管理（ゲームオーバー、駒選択中、移動先選択中）
- ・勝敗判定
- ・各駒特有の動き（移動が可能かどうか判定する）
- ・キャプチャの管理
- ・特殊ルールの実装（キャスリング、プロモーション、アンパサン）

3.2.2. クラス図

次の図のようにクラス、メンバ変数、関数を定義する。



3.3.3. 各関数の説明

<HardwareInput クラス>

- getMove(): MoveCommand

デバイスドライバのファイル(fd)を読み込み、割り込みがあるまで待機する。入力があれば、decodeCommand 関数を呼び出してその結果を返す。

- decodeCommand(uint32_t data): MoveCommand

受け取ったビット列を分解し、選択状態・ボタン状態・移動方向・時間切れフラグの情報を持つ MoveCommand として返す。

<ConsoleOutput クラス>

- drawBoard(Board board): void

盤面(Board クラス)を受け取り、駒の配置(getPiece)とカーソル位置(getCursorPos)と手番(currentTurn)をコンソールに出力する。

- drawResult(bool winner): void

白と黒どちらが勝ったかをコンソール出力する。

<Game クラス>

- start(): void

プログラム起動時に一度呼び出され、盤面の初期化 (board.initBoard())・ゲームの初期状態の設定を行う。

- play(): void

ゲームのメインループ。勝敗が決まる (checkWin) まで、input.getMove() の入力からコマンドを受け取り次第、コマンドによる処理(handleCommand)をおこない、display.drawBoard()で画面を出力しループする。

- switchTurn(): void

白と黒のターンを交互に入れ替える。

- checkWin(): bool

時間切れまたは、チェックメイトの判定を行い、勝敗がついたかどうかを判定する。

- handleCommand(MoveCommand cmd): void

コマンドを元に、Board クラスの関数の呼び出しによる盤面の操作および from の更新、手を打った(movePiece=true)ならターン変更(switchTurn)を行う。

<Board クラス>

- movePiece(Position from, Position to): bool

From から to へ駒を動かす処理。getPiece で駒を取得し、piece.canMove で駒の移動が可能かどうか判定し、可能なら square を更新し true を返す、不可能なら false を返す。

- getPiece(Position pos): Piece*

pos に存在する駒のクラスを返す。

- initBoard(): void

start で呼び出され、駒が初期位置に来るように square を更新する。

- isEmpty(): bool

GetPiece 関数で駒なしであった場合に true, 駒がある場合 false を返す。

- moveCursor(int direction): void

direction が意味する方向へ cursor を移動できるか判定し、可能なら移動。

- ・ getCursorPos(): Position

Cursor の Position を返す。

<Piece クラス>

- ・ canMove(Position from, Position to, Board& board): bool

From から to の移動がそのクラスに対応する駒で可能であることを返す。

- ・ getColor(): bool

その駒が白(1)か黒(0)かを返す。

- ・ getSymbol(): char

その駒を意味する文字を返す(display.drawBoard 用)。

3.3 検証方法 (担当：上治)

3.3.1 ハードウェアの単体テスト

Main モジュールの結果を led に出力して、ハードウェアへの入力とその出力が正しいことを確認する。カウントダウンを led に出力して、その挙動が正しいことを確認する。

3.3.2 ソフトウェアの単体テスト

入力は PC 上で与えられる状態（ドライバプログラムの作成）にして、入力に対してゲームシステムが期待する挙動をすることを確認する。特に、駒が有効な動きをすること、キャスリング・プロモーション・アンパサンの特殊ルールに対応していることを確認する。また、時間切れによるゲーム終了処理が行えることを確認する。

3.3.3 統合テスト

コンパイルして得られたチェスプログラムをハードウェアに差し込み、動かし、ハードウェアの入力に対応してコンソール上でチェスができることを確認する。

3.4 各自の担当

3.4.1 伊藤担当分 (担当：伊藤)

- ・FPGA 上の入力を変換しソフトウェアに返す回路の設計・制作
- ・タイマーの実装及び残り時間の表示をする回路の設計・制作

3.4.2 上治担当分 (担当：上治)

- ・チェスゲームアルゴリズムの実装
- ・main モジュールのアドレスをファイルに出力するデバイスドライバの作成
- ・単体テスト用のドライバプログラムの作成

4. スケジュール

第10回	第11回	第11回
コントロール層とエンティティ層の 実装、テスト	インターフェース層とデバイス ドライバの実装、テスト	統合テスト 修正・改良
回路及びモジュールの作成 基本的な状態遷移の確認	ソフトウェアとの協調のための修正	

4.1 ハードウェア（担当：伊藤）

・第 10 回（～6/17）

回路およびモジュールを設計・制作し、簡易的なソフトウェアを作成し実行することで、動作の確認を行う。

・第 11 回（～6/24）

ソフトウェア側の作成物と入出力がかみ合うように修正を加え、問題が生じた場合は議論し修正を行う。

・第 12 回（～7/1）

統合テストを行い、バグが残っていた場合は即座に修正する。制作物発表会に向けてスライドなどの準備を行う。

4.2 ソフトウェア（担当：上治）

・第 10 回（6/11～6/17）

チェスアプリのソフトウェア側の開発（コントロール層とエンティティ層と ConsoleOutput クラス）を行い、ユーザーインターフェース層（HardwareInput クラス）を PC 上の入出力で代替して動作するようにする。

・第 11 回（6/18～6/24）

main モジュールのデバイスドライバおよび、デバイスドライバのファイルを読み入出力を行うユーザーインターフェース層（HardwareInput クラス）を実装する

・第 12 回（6/25～7/1）

ソフトウェアとハードウェアの統合テスト、問題点の修正および制作物発表準備